

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\agent-a6a70c0115000b8c6.jsonl`  
- SHA-256: `cc1a40b4c199ac532752e1bfff1af1a29ecb3bb5b1d5db91d830ee46d229fe644`  
- Source modified: `2026-06-16T04:40:38+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `subagents`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T04:38:09.070Z)

Map the EXACT public APIs needed to build a video-free `replay\_fixture()` helper that reconstructs the deterministic rally-segmentation pipeline from a frozen trajectory CSV. Read files in the git worktree at: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\golden-fixtures\ (this is origin/master and INCLUDES the new #187 bounded-windowing ? read windowing.py from THIS path, not the main checkout).

Read and report EXACT signatures + return shapes for:

1. backend/pipeline/detectors/tracknet_runner.py ? `parse\_trajectory\_csv` (input columns Frame,Visibility,X,Y; return type/shape) AND `trajectory\_to\_action\_windows` (full signature, fps/frame_width params, what it returns, how it handles visible==False occlusion and fps<=0/width<=0 clamps).
2. backend/eval/windowing.py ? `build\_windows`, `WindowingPreset`, the SERVED and PROPOSAL presets: how a preset is constructed, its fields, how it's passed, `windows\_to\_intervals` if present, and WHAT #187 (bounded windowing) changed (e.g. a max_window_duration cap / inactivity split).
3. backend/eval/distill_local.py ? `build\_candidates` (full signature, what it returns: intervals + feature rows), `FEATURE\_NAMES` (the exact 5 shuttle feature keys).
4. backend/eval/experiment.py ? the `VideoCandidates` structure (fields), `predict()` (signature + what it returns), the `CONTROL` variant (how defined), `Variant` + `spec\_hash()`, `merge\_overlaps`, and the EXACT path from candidates ? predicted rally segments for the CONTROL variant (which must be PURE-STDLIB ? no numpy/classifier).
5. backend/eval/metrics.py ? `score()` (signature + the returned object's fields: precision/recall/f1/mean_iou/tp/fp/fn?) and `interval\_iou`, and the `Interval` type.
6. backend/eval/gt_loader.py ? `load\_gt\_intervals` (signature, strict vs lenient mode, return type).
7. backend/pipeline/detectors/rally_gate.py ? how `net\_crossings` is computed (is there a `gate\_windows`? an `estimate` param? `estimate\_play\_axis`/_median/_spread?).
8. backend/eval/fusion_compare.py and backend/eval/ablation.py ? how each `load\_videos`/_`load\_golden\_gts` reads the *_golden_features.json (which keys per candidate: shuttle/person/audio? does it read c['person'] UNCONDITIONALLY?).

Then produce a CONCRETE INTEGRATION RECIPE: the precise CONTROL-only call chain `parse\_trajectory\_csv ? trajectory\_to\_action\_windows/build\_windows(preset) ? build\_candidates ? rally\_gate net\_crossings ? experiment.predict(CONTROL) ? merge\_overlaps ? metrics.score(preds, gts, iou)`, with exact function names, argument lists, and return handling.

CRITICALLY, confirm against the code these red-team gotchas (answer YES/NO + file:line):

- (a) Do the existing loaders (fusion_compare/ablation/calibrate_wasb) read GT via a LENIENT loader that silently skips bad rows, vs the STRICT gt_loader.load_gt_intervals?
- (b) Does experiment `\_feature\_column` (or equivalent) silently default a MISSING feature column to 0.0 (warn only)?
- (c) Does any loader read `c['person']` UNCONDITIONALLY (so a person-stripped fixture raises KeyError)?
- (d) Does the CONTROL variant avoid numpy/LogisticRegression entirely (pure stdlib), and which variants are is_learned()?

Return signatures + the recipe + the 4 gotcha answers. Do NOT write any files ? this is read-only mapping.

2. assistant (2026-06-16T04:38:10.944Z)

I'll systematically map the exact APIs you need from the worktree. Let me start by exploring the golden fixtures directory structure and then read the critical files.

3. user (2026-06-16T04:38:11.507Z)

Exit code 2
/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''

4. assistant (2026-06-16T04:38:13.262Z)

Let me use PowerShell for the path with spaces:

5. user (2026-06-16T04:38:15.256Z)

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\golden-fixtures

Mode Name	LastWriteTime	Length
----	-----	-----
d----	6/16/2026 10:06 AM	.github
d----	6/16/2026 10:06 AM	backend
d----	6/16/2026 10:06 AM	docs
d----	6/16/2026 10:06 AM	eval_baselines
d----	6/16/2026 10:06 AM	scratch
d----	6/16/2026 10:06 AM	scripts
d----	6/16/2026 10:06 AM	tests
d----	6/16/2026 10:06 AM	tools
d----	6/16/2026 10:06 AM	training
-a---	6/16/2026 10:06 AM	611 .coveragerc
-a-h-	6/16/2026 10:06 AM	91 .git
-a---	6/16/2026 10:06 AM	924 .gitignore
-a---	6/16/2026 10:06 AM	4744 CLAUDE.md
-a---	6/16/2026 10:06 AM	2735 config.json
-a---	6/16/2026 10:06 AM	1628 mypy.ini
-a---	6/16/2026 10:06 AM	3359 pyproject.toml
-a---	6/16/2026 10:06 AM	245 pytest.ini
-a---	6/16/2026 10:06 AM	33528 README.md
-a---	6/16/2026 10:06 AM	629 requirements.txt
-a---	6/16/2026 10:06 AM	884 ruff.toml

6. assistant (2026-06-16T04:38:17.577Z)

Now let me read the critical files in sequence. I'll start with tracknet_runner.py:

7. user (2026-06-16T04:38:18.023Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
```

```

12         pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                    # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                    # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """

```

```

69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU'))))\"")
107         try:
108             res = self._wsl_bash(cmd)
109         except FileNotFoundError:
110             return False, "`wsl` not found ? is this running on Windows with WSL2
111             installed?"
112         except subprocess.TimeoutExpired:
113             return False, "WSL healthcheck timed out."
114         out = (res.stdout or "").strip()
115         if res.returncode != 0:
116             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117         return True, out
118
119 # ----- inference
120
121 def run_predict(self, video_win_path: str, output_win_dir: str,
122                 video_id: Optional[str] = None) -> Optional[str]:
123     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
124     None.
125
126     ``output_win_dir`` is a Windows directory (under the repo's output/) that
127     WSL writes into via its /mnt view, so the Windows process can read the CSV
128     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
129     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
130     a filename cannot collide on the output CSV.
131     """
132     if not self.cfg.weights_path:

```

```

131         logger.error(
132             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133             "in config.json to the WSL path of your trained TrackNetV4 weights."
134         )
135         return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}__{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         # "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                 "consider running tools/cleanup_caches.py.",
178                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194             self.cfg.timeout_sec)
195         return None

```

```

194
195         if res.returncode != 0:
196             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197             return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204             # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205             # regenerable intermediate). On earlier failure we return above without
cleaning.
206             if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207                 logger.info("Cache hygiene: removing staged video for %s "
208                             "(set indexing.tracknet.keep_frames=true to retain).", key)
209                 self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224         try:
225             res = self._wsl_bash(cmd)
226         except subprocess.TimeoutExpired:
227             logger.error("Staging video into WSL timed out.")
228             return None
229         if res.returncode != 0 or "OK" not in (res.stdout or ""):
230             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231             return None
232         return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252             points.sort(key=lambda p: p.frame)
253             return points

```

```

254
255 @staticmethod
256 def trajectory_to_action_windows(
257     points: List[TrajectoryPoint],
258     fps: float,
259     frame_width: float,
260     chunk_start: float = 0.0,
261     velocity_thresh: float = 0.02,
262     merge_gap: float = 2.0,
263     min_window_duration: float = 2.0,
264     chunk_index: Optional[int] = None,
265     max_window_duration: float = 0.0,
266     min_split_gap: float = 0.5,
267 ) -> List[Dict[str, Any]]:
268     """Convert a shuttle trajectory into candidate rally windows.
269
270     A frame is "active" when the shuttle is visible AND its inter-frame
271     displacement (normalised by frame width, per second) exceeds
272     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
273     ``merge_gap`` seconds of each other are grouped into one window; short
274     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
275     shape feeding the AI-handoff phase is identical.
276
277     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
278     against the *over-merge* failure where "shuttle moving" is conflated with "rally in
279     progress" and a single window swallows several rallies + the dead time between them. When
280     set, any window longer than this is recursively SPLIT at its largest internal inactivity
281     gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
282     provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
283     fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
284     recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
285     """
286     if fps <= 0:
287         fps = 30.0
288     if frame_width <= 0:
289         frame_width = 1280.0
290
291     # Compute per-frame normalised velocity from the last *visible* point.
292     active = [] # (frame_idx, velocity)
293     prev: Optional[TrajectoryPoint] = None
294     for p in points:
295         if not p.visible:
296             prev = None # break the trajectory; absent shuttle = not in flight
297             continue
298         if prev is not None:
299             dframes = p.frame - prev.frame
300             if dframes > 0:
301                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
302                 velocity = (dist / frame_width) * (fps / dframes)
303                 if velocity > velocity_thresh:
304                     active.append((p.frame, velocity))
305             prev = p
306
307     if not active:
308         return []
309
310     max_gap_frames = int(merge_gap * fps)
311

```

```

312     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
313         vels = [v for _, v in group]
314         return {
315             "start": group[0][0] / fps + chunk_start,
316             "end": group[-1][0] / fps + chunk_start,
317             "active_frame_count": len(group),
318             "peak_velocity": max(vels),
319             "mean_velocity": sum(vels) / len(vels),
320             "track_id_count": 1, # single shuttle; kept for window-shape parity
321             "chunk_index": chunk_index,
322         }
323
324     groups: List[List[Tuple[int, float]]] = []
325     group = [active[0]]
326     for af in active[1:]:
327         if af[0] - group[-1][0] <= max_gap_frames:
328             group.append(af)
329         else:
330             groups.append(group)
331             group = [af]
332     groups.append(group)
333
334     if max_window_duration > 0:
335         groups = TrackNetRunner._split_overlong_groups(
336             groups, int(max_window_duration * fps), int(min_split_gap * fps))
337
338     windows = [_finalize(g) for g in groups]
339     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
340
341     @staticmethod
342     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
343                               min_split_frames: int) -> List[List[Tuple[int, float]]]:
344         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
345         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
346         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
347         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
348         rally). Iterative worklist to bound stack depth."""
349         if max_win_frames <= 0:
350             return groups
351         out: List[List[Tuple[int, float]]] = []
352         work = list(groups)
353         while work:
354             g = work.pop()
355             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
356                 out.append(g)
357                 continue
358             best_i, best_gap = -1, -1
359             for i in range(1, len(g)):
360                 gap = g[i][0] - g[i - 1][0]
361                 if gap > best_gap:
362                     best_gap, best_i = gap, i
363             if best_i < 0 or best_gap < min_split_frames:
364                 out.append(g) # too long but no real dead-time gap ? keep as one rally
365                 continue
366             work.append(g[:best_i])
367             work.append(g[best_i:])
368         out.sort(key=lambda grp: grp[0][0])
369         return out
370

```

8. assistant (2026-06-16T04:38:20.110Z)

Now let me read windowing.py:

9. user (2026-06-16T04:38:20.700Z)

```
1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4      ?
5      `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6      built
7      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9      pipeline
10     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11     merge_gap=2.0,
12     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13     is
14     measured on candidates **production never serves**, so a "win" need not transfer. This
15     is
16     exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
17     set
18     (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
21     the
22     SERVED preset is **single-sourced from the same Pydantic config defaults production
23     reads**
24     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25     Helpers
26     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
27     *actual* served windowing out of a live config (overrides included), so an A/B can
28     target the
29     operating point a given deployment truly serves.
30
31     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
32     windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
33     `per_user_gate`). This module is the **what windowing** ; that one is the **promotion
34     gate**.
35
36     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
37     a
38     caller opts in (`distill_local` re-points its module-level presets here, byte-
39     identically).
40
41     """
42     from __future__ import annotations
43
44     from dataclasses import dataclass
45     from typing import Any, Dict, List, Mapping, Tuple
46
47     from backend.config.models import IndexingConfig
48
49     Interval = Tuple[float, float]
50
51     __all__ = [
52         "WindowingPreset",
53         "SERVED",
54         "PROPOSAL",
55         "PRESETS",
56         "served_windowing_from_config",
57         "build_windows",
58         "windows_to_intervals",
59     ]
```

```

46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset is the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64
65         def as_tuple(self) -> Tuple[float, float, float]:
66             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
67             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
68             bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
69             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
70
71         def to_dict(self) -> Dict[str, Any]:
72             return {
73                 "name": self.name,
74                 "velocity_thresh": self.velocity_thresh,
75                 "merge_gap": self.merge_gap,
76                 "min_window_duration": self.min_window_duration,
77                 "max_window_duration": self.max_window_duration,
78             }
79
80
81     def _served_preset_from_defaults() -> WindowingPreset:
82         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
83
84         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
85         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
86         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
87         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
88
89         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
90         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
91         so the two can never silently diverge again.
92         """
93         idx = IndexingConfig() # construct with defaults ? the served operating point
94         return WindowingPreset(
95             name="served",
96             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet

```

```

default 0.02
97         merge_gap=float(idx.dead_time_merge_gap),
98         min_window_duration=float(idx.min_action_window_duration),
99         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
100     )
101
102
103     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
104     #: only promotable if it holds up HERE ? the operating point real users get.
105     SERVED: WindowingPreset = _served_preset_from_defaults()
106
107     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
108     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
109     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
110     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
111     PROPOSAL: WindowingPreset = WindowingPreset(
112         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
113     )
114
115     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
116     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
117
118
119     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
120         """The SERVED windowing a *specific* live config serves (overrides honoured).
121
122         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
123         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
124         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
125         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
126         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
127         """
128         idx = _as_mapping(config, "indexing")
129         family_cfg = _as_mapping(idx, family)
130         return WindowingPreset(
131             name=f"served:{family}",
132             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
133             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
134             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
135             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
136         )
137
138
139     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
140         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
141         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
142         if obj is None:
143             return {}
144         if isinstance(obj, Mapping):
145             sub = obj.get(key, {})

```

```

146         elif hasattr(obj, "get"):
147             sub = obj.get(key, {})
148         else:
149             sub = getattr(obj, key, {})
150         return sub if isinstance(sub, Mapping) else {}
151
152
153     def build_windows(points: List[Any], fps: float, frame_width: float,
154                       preset: WindowingPreset) -> List[Dict[str, Any]]:
155         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
156         call site).
157         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
158         named preset
159         instead of three loose floats, so eval candidate-building and the served path
160         reference the
161         SAME knobs by name. Imported lazily to keep this module import-light (no detector
162         deps)."""
163         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
164         vt, mg, mwd = preset.as_tuple()
165         return TrackNetRunner.trajectory_to_action_windows(
166             points, fps=fps, frame_width=frame_width,
167             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
168             max_window_duration=preset.max_window_duration)
169
170     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
171         """``[[start, end, ...]]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
172         speak."""
173         return [(w["start"], w["end"]) for w in windows]

```

10. assistant (2026-06-16T04:38:22.606Z)

Now let me read distill_local.py:

11. user (2026-06-16T04:38:23.347Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21     python -m backend.eval.distill_local --manifest videos.json --model-out
22     output/local_rally_model.json
23     """
24     from __future__ import annotations
25     import json

```

```

26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37     from backend.eval import windowing as _windowing
38
39     Interval = Tuple[float, float]
40
41     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
42 "net_crossings"]
43     # Recall-oriented proposal: deliberately permissive so real rallies are among the
44     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
45     # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
46     that
47     # module's docstring for the eval?serve bug these named presets close. The bare tuples
48     are
49     # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
50     # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
51     PROPOSAL = _windowing.PROPOSAL.as_tuple()
52     BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
53     learning
54
55
56
57     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
58                         proposal: Tuple[float, float, float] = PROPOSAL) ->
59     Tuple[List[Interval], List[List[float]]]:
60     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
61     rows)."""
62     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
63     vt, mg, mwd = proposal
64     wins = TrackNetRunner.trajectory_to_action_windows(
65         points, fps=fps, frame_width=frame_width,
66         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
67     intervals = [(w["start"], w["end"]) for w in wins]
68     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
69     crossings
70     X: List[List[float]] = []
71     for w, g in zip(wins, gated):
72         dur = max(1e-6, w["end"] - w["start"])
73         win_frames = max(1.0, dur * fps)
74         X.append([
75             dur, # rally length (sec) ?
76             float(w["peak_velocity"]), # already normalized by
77             float(w["mean_velocity"]),
78             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
79             float(g.crossings), # net crossings (count) ?
80             the rally signal
81         ])
82     return intervals, X
83
84
85
86     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
87     List[Interval]:
88     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
89     vt, mg, mwd = BASELINE_LOCAL

```

```

79     wins = TrackNetRunner.trajectory_to_action_windows(
80         points, fps=fps, frame_width=frame_width,
81         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82     return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)
90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
91             X,
92             "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
93             fw)}
94
95     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
96     List[Interval],
97     threshold: float = 0.5) -> List[Interval]:
98         if not intervals:
99             return []
100         proba = model.predict_proba(X)
101         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
102         return merge_overlaps(pos, gap_tol=1.0)
103
104     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
105     model_out: Optional[str] = None) -> dict:
106         videos = [load_video(s, iou) for s in specs]
107         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
108             f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
109         for v in videos:
110             ceil = score(v["intervals"], v["gts"], iou).recall
111             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
112                 f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
113
114         result: dict = {"videos": [v["name"] for v in videos]}
115         if len(videos) >= 2:
116             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
117             out) ---")
118             clf, base = [], []
119             for i, held in enumerate(videos):
120                 others = [v for j, v in enumerate(videos) if j != i]
121                 X = [row for v in others for row in v["X"]]
122                 y = [t for v in others for t in v["y"]]
123                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
124                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
125                 sc = score(preds, held["gts"], iou)
126                 bsc = score(held["baseline"], held["gts"], iou)
127                 clf.append(sc)
128                 base.append(bsc)
129                 print(f"    held-out {held['name']}: LEARNED P={sc.precision:.3f}
130                 R={sc.recall:.3f} F1={sc.f1:.3f}"
131                     f"    | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
132                     F1={bsc.f1:.3f}")
133             def mean(xs, a):
134                 return sum(getattr(s, a) for s in xs) / len(xs)
135             print(f"    MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
136                 R={mean(clf, 'recall'):.3f} "
137                     f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
138                     f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
139             result["mean_learned_f1"] = mean(clf, "f1")
140             result["mean_baseline_f1"] = mean(base, "f1")

```

```

137     # Final model on ALL videos (for production / running on a fresh held-out video).
138     Xall = [row for v in videos for row in v["X"]]
139     yall = [t for v in videos for t in v["y"]]
140     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141     assert final.w is not None # set by fit()
142     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144     if model_out:
145         final.save(model_out)
146         print(f"final model (all videos) -> {model_out}")
147         result["model_out"] = model_out
148     return result
149
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166

```

12. assistant (2026-06-16T04:38:25.610Z)

Now let me read experiment.py:

13. user (2026-06-16T04:38:26.409Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare` ? labeled A/B vs golden labels: per-video + LOVO-honest

```

```

26     aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27     held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29     variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30     human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32     (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
56     ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
58     scores
59
60     logger = logging.getLogger(__name__)
61
62     # Where a comparison run appends one reproducible record. Tracked location (NOT under
63     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64     # regression.DEFAULT_BASELINE_PATH.
65     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66     _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73     # ----- Variant
74
75     @dataclass
76     class Variant:
77         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
78
79         A variant turns one video's persisted candidate windows + features into rally
80         intervals. Every knob defaults to the **control** behaviour (no gate, no learned
81         filter), so a variant that merely *carries* a new, disabled cue is byte-identical
82         to control ? the core no-regression guarantee.
83
84         Fields:
85         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
86         for the leaderboard and for selecting the served path (the existing
87         `segmenter_registry` + `IndexingConfig` do the actual selection in production).

```

```

89         New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md`` §3.1).
94     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100    - ``feature_names`` ? the persisted features the learned filter consumes. When set
101      WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102      (honest) ? the recipe, not a pinned artifact.
103    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104      overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105    """
106
107    name: str
108    segmenter: str = "wasb_hybrid"
109    config_overrides: Dict[str, Any] = field(default_factory=dict)
110    cue_flags: Dict[str, bool] = field(default_factory=dict)
111    min_crossings: int = 0
112    min_players: int = 0
113    classifier_model: Optional[str] = None
114    feature_names: Optional[List[str]] = None
115    threshold: float = 0.5
116    merge_gap: float = 1.0
117    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119    # failure where a fixed 0.5 zeroed out a small-candidate video.
120    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121    calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123    def is_learned(self) -> bool:
124        """True if this variant applies a learned classifier (pinned model or
recipe)."""
125        return self.classifier_model is not None or bool(self.feature_names)
126
127    def to_dict(self) -> dict:
128        return {
129            "name": self.name,
130            "segmenter": self.segmenter,
131            "config_overrides": self.config_overrides,
132            "cue_flags": self.cue_flags,
133            "min_crossings": self.min_crossings,
134            "min_players": self.min_players,
135            "classifier_model": self.classifier_model,
136            "feature_names": self.feature_names,
137            "threshold": self.threshold,
138            "merge_gap": self.merge_gap,
139            "tune_threshold": self.tune_threshold,
140            "calibrate": self.calibrate,
141        }
142
143    @classmethod
144    def from_dict(cls, d: dict) -> "Variant":
145        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146        return cls(**{k: v for k, v in d.items() if k in known})
147
148    def spec_hash(self) -> str:

```

```

149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with ``load_video_candidates``, or directly in tests.
172         """
173
174         name: str
175         intervals: List[Interval]
176         features: Dict[str, List[float]] = field(default_factory=dict)
177         gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                             name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
196     List[List[float]]:
197         cols = [_feature_column(vc, name) for name in feature_names]
198         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask

```

```

211
212
213 def predict(variant: Variant, vc: VideoCandidates,
214             model: Optional[LogisticRegression] = None,
215             threshold: Optional[float] = None,
216             calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226     """
227     mask = _gate_mask(variant, vc)
228     if variant.feature_names:
229         if model is None:
230             if variant.classifier_model is None:
231                 raise ExperimentError(
232                     f"variant {variant.name!r} is learned but has no model to predict "
233                     f"with (pass a fitted model or set classifier_model)")
234             model = LogisticRegression.load(variant.classifier_model)
235             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
236 variant.feature_names))]
237             if calibrator is not None:
238                 proba = [calibrator(p) for p in proba]
239                 thr = variant.threshold if threshold is None else threshold
240                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
241             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
242             return merge_overlaps(kept, gap_tol=variant.merge_gap)
243
244 def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
245                       train_videos: Sequence[VideoCandidates], iou: float
246                       ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
247     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
248     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
249     (use the variant default). Honest: never looks at the held-out video.
250     """
251     calibrator: Optional[Callable[[float], float]] = None
252     if variant.calibrate:
253         raw: List[float] = []
254         y: List[int] = []
255         for vc in train_videos:
256             raw.extend(float(p) for p in
257 model.predict_proba(_feature_matrix(vc, variant.feature_names or
258 [])))
259             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
260         if variant.calibrate == "platt":
261             calibrator = fit_platt(raw, y)
262         elif variant.calibrate == "isotonic":
263             calibrator = fit_isotonic(raw, y)
264         else:
265             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
266 'platt'/'isotonic')")
267     threshold: Optional[float] = None
268     if variant.tune_threshold:
269         best_t, best_f1 = variant.threshold, -1.0
270         for k in range(1, 20):
271             # grid 0.05..0.95 on the train
272             t = round(0.05 * k, 2)
273             f1s = [score(predict(variant, vc, model=model, threshold=t,

```

```

calibrator=calibrator),
270             vc.gts or [], iou).f1 for vc in train_videos]
271         mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272         if mean_f1 > best_f1:
273             best_f1, best_t = mean_f1, t
274         threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300

```

14. assistant (2026-06-16T04:38:28.570Z)

Let me continue reading experiment.py:

15. user (2026-06-16T04:38:29.568Z)

```

300
301     Prediction policy:
302     - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304     - **learned, pinned model** (``classifier_model`` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307     - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309     - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310
311     for vc in videos:
312         if vc.gts is None:
313             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315     per_video: List[Dict[str, Any]] = []
316     predictions: Dict[str, List[Interval]] = {}
317
318     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None

```

```

319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:

```

```

374         f1k[ov].append(got[ov])
375         ratios.append(_segm.segment_count_ratio(preds, gts))
376
377     def _mean(xs: List[float]) -> float:
378         return sum(xs) / len(xs) if xs else 0.0
379     out: Dict[str, Any] = {"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380     out["segment_count_ratio"] = _mean(ratios)
381     return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}

```

```

425
426 by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427 per_video_delta = [
428     {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429     for p in eval_b["per_video"]]
430 ]
431
432 # One fingerprint over the whole label set ? detects "the deltas were measured
against
433 # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434 all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436 result: Dict[str, Any] = {
437     "iou": iou,
438     "label_set_hash": gt_hash(all_gts),
439     "num_videos": len(videos),
440     "variant_a": eval_a,
441     "variant_b": eval_b,
442     "delta": delta,
443     "delta_pct": delta_pct,
444     "per_video_delta": per_video_delta,
445 }
446 if honest_stats:
447     result["honest"] = honest_summary(videos, eval_a, eval_b)
448 return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455     agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471     """
472     preds_a = predict(variant_a, video)
473     preds_b = predict(variant_b, video)
474     mr = match_intervals(preds_a, preds_b, agree_iou)
475
476     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477               for m in mr.matches]
478     agree_iou = [m.iou for m in mr.matches]
479     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482     def _dur(ivs: List[Interval]) -> float:
483         return sum(e - s for s, e in ivs)
484
485     return {
486         "video": video.name,
487         "agree_iou": agree_iou,

```

```

487         "variant_a": variant_a.name,
488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499 
```

16. assistant (2026-06-16T04:38:32.060Z)

Now let me read metrics.py:

17. user (2026-06-16T04:38:32.795Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a
22        b_start, b_end = b
23        if a_end <= a_start or b_end <= b_start:
24            return 0.0
25        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26        if inter <= 0.0:
27            return 0.0
28        union = (a_end - a_start) + (b_end - b_start) - inter
29        return inter / union if union > 0 else 0.0
30
31
32    @dataclass
33    class Match:
34        """One matched predicted?ground-truth pair."""
35        pred_index: int
36        gt_index: int
37        iou: float
38
39
40    @dataclass
41    class MatchResult:
42        """Outcome of matching predictions to ground truth at a given IoU threshold."""
43        matches: List[Match] = field(default_factory=list)
44        unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45        unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives

```

```

(misses)
46
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                     iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62             # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63             if iou > 0.0 and iou >= iou_threshold:
64                 candidates.append((iou, pi, gi))
65     # Highest IoU first; ties broken by indices for determinism.
66     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                       unmatched_pred_indices=unmatched_pred,
81                       unmatched_gt_indices=unmatched_gt)
82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float # over matched pairs (0.0 if none)
97     mean_start_error: float # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104 def score(preds: List[Interval], gts: List[Interval],
105          iou_threshold: float = 0.5,
106          match_result: Optional[MatchResult] = None) -> Score:
107     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108

```

```

109     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110     extracting the failure lists); otherwise matching is done here.
111     """
112     mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)
113
114     tp = len(mr.matches)
115     fp = len(mr.unmatched_pred_indices)
116     fn = len(mr.unmatched_gt_indices)
117
118     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
121
122     if mr.matches:
123         mean_iou = sum(m.iou for m in mr.matches) / tp
124         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
125         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
126     else:
127         mean_iou = mean_start_err = mean_end_err = 0.0
128
129     return Score(
130         iou_threshold=iou_threshold,
131         num_pred=len(preds), num_gt=len(gts),
132         true_positives=tp, false_positives=fp, false_negatives=fn,
133         precision=precision, recall=recall, f1=f1,
134         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
135     )
136
137
138 def pr_curve(preds: List[Interval], gts: List[Interval],
139             thresholds: Optional[List[float]] = None) -> List[Score]:
140     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142     Lets you see strict-vs-lenient behaviour rather than a single operating point.
143     """
144     if thresholds is None:
145         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146     return [score(preds, gts, iou_threshold=t) for t in thresholds]
147

```

18. assistant (2026-06-16T04:38:34.567Z)

Now let me read gt_loader.py:

19. user (2026-06-16T04:38:35.276Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - `annotations.load_annotations`: positional `start,end` (the collector's
5       `curate export` format), raises on bad rows;
6     - `calibrate_wasb.load_golden_gts`: DictReader requiring `start_time/end_time`
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and `#` comments ignored):

```

```

15 - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16 columns ? rally_number, ending_reason, sport, ? ? are ignored);
17 - no header: columns 0,1 positionally.
18 Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19 (``annotations.parse_timestamp``).
20 """
21 import csv
22 import logging
23 from typing import List, Optional, Tuple
24
25 from backend.eval.annotations import parse_timestamp
26
27 logger = logging.getLogger(__name__)
28
29 Interval = Tuple[float, float]
30
31 _START_NAMES = ("start", "start_time")
32 _END_NAMES = ("end", "end_time")
33
34 def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35     """If `row` is a header naming start/end columns, return their indices, else
36     None."""
37     cells = [(c or "").strip().lower() for c in row]
38     start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39     end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40     if start_idx is not None and end_idx is not None:
41         return start_idx, end_idx
42     return None
43
44 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45     """Load GT rally intervals from either golden-CSV convention.
46
47     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50     skipped ? a silently shrinking GT set flattens precision.
51
52     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53     (they break one-to-one matching and usually indicate a labeling error).
54     """
55     intervals: List[Interval] = []
56     skipped = 0
57     cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                     continue # consume the header row
70             cols = (0, 1) # headerless: positional start,end
71             assert cols is not None # every branch above assigns it
72             si, ei = cols
73             try:
74                 if len(row) <= max(si, ei):
75                     raise ValueError(f"expected at least {max(si, ei) + 1} columns, got {len(row)}")
76                 start = parse_timestamp(row[si])

```

```

77         end = parse_timestamp(row[ei])
78         if start < 0:
79             raise ValueError(f"negative start ({start})")
80         if start >= end:
81             raise ValueError(f"start ({start}) >= end ({end})")
82     except ValueError as e:
83         if strict:
84             raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85         skipped += 1
86         continue
87     intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                 "labeling error? Overlaps break one-to-one matching.",
97                 csv_path, s1, e1, s2, e2)
98     return intervals
99

```

20. assistant (2026-06-16T04:38:37.274Z)

Now let me read rally_gate.py:

21. user (2026-06-16T04:38:38.178Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool

```

```

35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed

```

```

99     `(axis, net, span)` (from `estimate_play_axis`) so the  $O(P \log P)$  estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

22. assistant (2026-06-16T04:38:40.734Z)

Now let me read fusion_compare.py and ablation.py to check the loader patterns:

23. user (2026-06-16T04:38:41.501Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6     designed seam): the person features are extra columns the learned LogisticRegression
weights,
7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9     Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}

```

```

30     by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31     vids = [p["video"] for p in res["variant_a"]["per_video"]]
32     return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35 def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36     """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37     feature columns + golden gts). The harness derives labels from gts itself."""
38     videos: List[experiment.VideoCandidates] = []
39     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40         with open(path, encoding="utf-8") as f:
41             d = json.load(f)
42             cands = d["candidates"]
43             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44             features: Dict[str, List[float]] = {}
45             for fn in SHUTTLE_FEATURE_NAMES:
46                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
47             for fn in PERSON_FEATURE_NAMES:
48                 features[fn] = [float(c["person"][fn]) for c in cands]
49             gts = [(float(a), float(b)) for a, b in d["gts"]]
50             videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52     return videos
53
54
55 def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56             threshold: float = 0.5) -> Dict[str, Any]:
57     shuttle_only = experiment.Variant(name="shuttle_only",
58                                       feature_names=list(SHUTTLE_FEATURE_NAMES),
59                                       threshold=threshold)
60     fusion = experiment.Variant(name="shuttle_person_fusion",
61                                 feature_names=list(SHUTTLE_FEATURE_NAMES) +
62                                 list(PERSON_FEATURE_NAMES),
63                                 threshold=threshold)
64     return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66 def format_result(res: Dict[str, Any], iou: float) -> str:
67     a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68     ds = delta_stats(res)
69     st = ds["sign_test"]
70     lines = [
71         f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
72         ===",
73         f"    shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
74         IoU={a['mean_iou']:.3f}",
75         f"    +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
76         IoU={b['mean_iou']:.3f}",
77         f"    ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
78         ?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
79         f"    paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
80         {ds['ci_high']:+.3f}]  "
81         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
82         f"| CI clears 0: {ds['ci_excludes_zero']}",
83         f"    per-video ?F1:",
84     ]
85     for pv in res["per_video_delta"]:
86         lines.append(f"        {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
87     return "\n".join(lines)
88
89
90 def main() -> None:
91     ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
92     shuttle+person (golden set).")
93     ap.add_argument("--features-dir", default="output")

```

```

87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

24. assistant (2026-06-16T04:38:43.567Z)

Now let me check ablation.py:

25. user (2026-06-16T04:38:44.439Z)

```

1     """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3     A *systematic, repeatable* version of the one-off ablation
(`scratch/gate_ab_ablation.py`):
4     run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
5     signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
6     95% CI + paired sign test + the C4 over-segmentation ratio. See
`scratch/ABLATION_FRAMEWORK_PLAN.md`
7     (the design) and `docs/ABLATION_LAYER.md` (the shipped note).
8
9     **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
10    `experiment.compare` + `eval_stats` engine, exactly as `compare` was a driver over
11    `metrics`/`calibration`. For each signal `s` the marginal contribution is:
12
13    ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop, ?F1
primary
14
15    operationally `experiment.compare(videos, variant_a=without_s, variant_b=full)` ? A is
the
16    ablated (smaller) variant, B is full, so the reported B-A delta IS ?(s), a per-video-
paired ?F1
17    with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19    Two ablation granularities fall out of one inventory:
20
21    - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22    - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23    "is this old SIGNAL still relevant?" question at cue granularity).
24
25    A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26    a **gate knob** (Gate-A's `min_crossings` threshold ? ablated by zeroing the knob, NOT
by
27    dropping a column). The runner handles both uniformly: `without_s` subtracts the
columns AND/OR

```

```

28     zeroes the gate knob from ``cfg.full``.
29
30     Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; §6 of the
plan):
31     - never a bare mean ? every row ships a CI + the exact sign test;
32     - a signal earns its keep only when removal drops F1 with CI excluding 0 AND the
sign test
33     agreeing** (the ``per_user_gate`` bar, in reverse);
34     - structural signals are exempt from auto-demotion ? a guard whose value is in
failure modes
35     the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36     flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37     but tags ``structural_protected``;
38     - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39     MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41     Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42     writes one ``<name>_golden_features.json`` per video). CLI::
43
44     python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45     """
46     from __future__ import annotations
47
48     import argparse
49     import glob
50     import json
51     import logging
52     import os
53     from dataclasses import dataclass, field
54     from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56     from backend.eval import experiment
57     from backend.eval.experiment import Variant, VideoCandidates
58
59     logger = logging.getLogger(__name__)
60
61     # --- The signal inventory (the things the runner ablates), grouped by producer.
-----
62     # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63     # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64     # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65     SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                         "active_ratio", "net_crossings")
67     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                         "in_court_ratio", "shuttle_player_colocation")
69     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
70
71     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
72     EARNS_KEEP = "earns_keep" # CI clears 0 AND sign test agrees ? removing it
hurts
73     SUGGESTIVE = "suggestive" # point estimate leans positive but CI spans 0
74     INERT = "inert" # wash this run (?0, CI spans 0) ? candidate,
not proof
75     STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
lift"
76

```

```

77 #: Human-readable one-liners for each verdict (used in the PDF + table legend).
78 VERDICT_LABEL = {
79     EARNS_KEEP: "clears-bar",
80     SUGGESTIVE: "suggestive",
81     INERT: "inert",
82     STRUCTURAL_PROTECTED: "structural-protected",
83 }
84
85
86 @dataclass(frozen=True)
87 class Signal:
88     """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
89
90     - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91     - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
whole group.
92     - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
93     pure gate signal like Gate-A whose column is shared with the shuttle group).
94     - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95     the full variant rather than dropping a column.
96     - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97     flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98     """
99
100     name: str
101     group: str
102     columns: Tuple[str, ...] = ()
103     gate_knob: Optional[str] = None
104     structural: bool = False
105
106
107 @dataclass
108 class AblationConfig:
109     """How to run an ablation over a corpus.
110
111     - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112     the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113     ``feature_names`` = all columns and the gates ON.
114     - ``signals`` ? the inventory to ablate.
115     - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116     - ``iou`` ? temporal-IoU match threshold.
117     - ``honest`` ? LOVO for learned variants (the number to trust).
118     - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119     - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120     """
121
122     full: Variant
123     signals: List[Signal]
124     granularity: str = "group"
125     iou: float = 0.5
126     honest: bool = True
127     seed: int = 0
128     p_threshold: float = 0.05
129
130
131 @dataclass
132 class AblationRow:
133     """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a

```

```

verdict)."""
134
135     signal: str
136     group: str
137     structural: bool
138     dropped_columns: List[str]
139     gate_knob: Optional[str]
140     n: int
141     delta_f1: float
142     ci_low: float
143     ci_high: float
144     ci_excludes_zero: bool
145     sign_wins: int
146     sign_losses: int
147     sign_p_value: float
148     f1_full: float
149     f1_ablated: float
150     seg_ratio_full: float
151     seg_ratio_ablated: float
152     verdict: str
153     structural_protected: bool
154
155     def as_dict(self) -> Dict[str, Any]:
156         return {
157             "signal": self.signal,
158             "group": self.group,
159             "structural": self.structural,
160             "dropped_columns": list(self.dropped_columns),
161             "gate_knob": self.gate_knob,
162             "n": self.n,
163             "delta_f1": self.delta_f1,
164             "ci_low": self.ci_low,
165             "ci_high": self.ci_high,
166             "ci_excludes_zero": self.ci_excludes_zero,
167             "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                          "p_value": self.sign_p_value},
169             "f1_full": self.f1_full,
170             "f1_ablated": self.f1_ablated,
171             "segment_count_ratio_full": self.seg_ratio_full,
172             "segment_count_ratio_ablated": self.seg_ratio_ablated,
173             "verdict": self.verdict,
174             "structural_protected": self.structural_protected,
175         }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
181         result."""
182         granularity: str
183         iou: float
184         num_videos: int
185         honest_lovo: bool
186         full_set_signals: List[str]
187         full_spec_hash: str
188         label_set_hash: str
189         rows: List[AblationRow] = field(default_factory=list)
190
191         def sorted_rows(self) -> List[AblationRow]:
192             """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
193             order)."""
194             return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
195
196         def as_dict(self) -> Dict[str, Any]:

```

```

196         return {
197             "granularity": self.granularity,
198             "iou": self.iou,
199             "num_videos": self.num_videos,
200             "honest_lovo": self.honest_lovo,

```

26. assistant (2026-06-16T04:38:46.413Z)

Let me check where ablation reads the golden features:

27. user (2026-06-16T04:38:47.333Z)

```

200         "honest_lovo": self.honest_lovo,
201         "full_set_signals": list(self.full_set_signals),
202         "full_spec_hash": self.full_spec_hash,
203         "label_set_hash": self.label_set_hash,
204         "rows": [r.as_dict() for r in self.sorted_rows()],
205     }
206
207
208     # ----- the driver
209
210
211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
212         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
213         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
214         mutates ``full``."""
215         cols = set(columns)
216         fn = full.feature_names
217         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219         if gate_knob == "min_crossings":
220             overrides["min_crossings"] = 0
221         elif gate_knob == "min_players":
222             overrides["min_players"] = 0
223         elif gate_knob is not None:
224             raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227                                             "name": f"?{dropped_label}")
228
229
230     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231                 sign_p: float, p_threshold: float) -> Tuple[str, bool]:
232         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
233
234         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238         if structural:
239             return STRUCTURAL_PROTECTED, True
240         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241         if ci_excludes_zero and sign_agrees:
242             return EARNS_KEEP, False
243         if sign_wins > sign_losses:
244             # leans toward "removal hurts" but evidence is
thin

```

```

244         return SUGGESTIVE, False
245     return INERT, False
246
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                   signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252         Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
253         ablated variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
254         ?(s). Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
255         stats come for free."""
256         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                                honest_stats=True)
259         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
260         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261         from backend.eval.eval_stats import paired_delta_ci
262         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263         by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264         vids = [p["video"] for p in res["variant_a"]["per_video"]]
265         pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
266                             seed=cfg.seed)
267         st = pd["sign_test"]
268         sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
269         sign_p = float(st["p_value"])
270         verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
271                                       sign_wins, sign_losses, sign_p, cfg.p_threshold)
272
273         seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
274         seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
275
276         return AblationRow(
277             signal=signal.name,
278             group=signal.group,
279             structural=signal.structural,
280             dropped_columns=list(signal.columns),
281             gate_knob=signal.gate_knob,
282             n=int(pd["n"]),
283             delta_f1=float(pd["mean_delta"]),
284             ci_low=float(pd["ci_low"]),
285             ci_high=float(pd["ci_high"]),
286             ci_excludes_zero=bool(pd["ci_excludes_zero"]),
287             sign_wins=sign_wins,
288             sign_losses=sign_losses,
289             sign_p_value=sign_p,
290             f1_full=float(res["variant_b"]["aggregate"]["f1"]),
291             f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
292             seg_ratio_full=float(seg_full),
293             seg_ratio_ablated=float(seg_abl),
294             verdict=verdict,
295             structural_protected=protected,
296         )
297
298     def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299         """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
300         columns,

```

Let me find where ablation loads the golden features:

29. user (2026-06-16T04:38:50.120Z)

```
386: def load_videos(features_dir: str) -> List[VideoCandidates]:
```

30. user (2026-06-16T04:38:52.511Z)

```
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395                 cands = d["candidates"]
396                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397                 features: Dict[str, List[float]] = {}
398                 for fn in SHUTTLE_COLUMNS:
399                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
400                 for fn in PERSON_COLUMNS:
401                     features[fn] = [float(c["person"][fn]) for c in cands]
402                 for fn in AUDIO_COLUMNS:
403                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404                 gts = [(float(a), float(b)) for a, b in d["gts"]]
405                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
406                                             features=features, gts=gts))
407         return videos
408
409
410     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
411         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
412         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
413         (distinct from the static Gate-A litmus)."""
414         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
415         return experiment.Variant(name="full_learned", feature_names=cols,
416                                 threshold=threshold, min_crossings=min_crossings)
417
418
419     # ----- table
rendering
420
421
422     def format_table(report: AblationReport) -> str:
423         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
424         rows = report.sorted_rows()
425         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
426                 f" . n={report.num_videos} . IoU {report.iou}"
427                 f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
428                 f" . {report.granularity.upper()}")
429         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
430         widths = [18, 17, 18, 14, 5, 22]
431
432         def _fmt_row(cells: Sequence[str]) -> str:
433             return " ".join(c.ljust(w) for c, w in zip(cells, widths))
434
435         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
```

31. assistant (2026-06-16T04:38:54.480Z)

Now let me check `calibrate_wasb.py` for the loader pattern:

32. user (2026-06-16T04:38:55.614Z)

```
33: def load_golden_gts(csv_path: str) -> List[Interval]:
41:     return load_gt_intervals(csv_path, strict=False)
```

33. assistant (2026-06-16T04:38:57.731Z)

Now let me check the `classifier.py` for the `LogisticRegression` signature:

34. user (2026-06-16T04:38:58.227Z)

```
1      """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3      The offline learned segmenter (ADR-004) must run locally with no external API and
4      no heavy ML stack ? the project already depends on numpy, and sklearn is not
5      installed. This implements just enough: standardized features, balanced-class
6      gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7      model is a small, human-readable, version-controllable file.
8
9      It deliberately stays simple: the rally-window features are few and roughly
10     linearly separable (high player motion + sustained activity => real rally), so a
11     well-regularized linear model is a sensible, inspectable baseline. Swap in a
12     richer model later behind the same fit/predict_proba/save/load interface.
13     """
14
15     import json
16     from typing import List, Optional
17
18     import numpy as np
19
20
21     class LogisticRegression:
22         def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23             self.lr = lr
24             self.epochs = epochs
25             self.l2 = l2
26             self.w: Optional[np.ndarray] = None
27             self.b: float = 0.0
28             self.mean: Optional[np.ndarray] = None
29             self.std: Optional[np.ndarray] = None
30             self.feature_names: Optional[List[str]] = None
31
32         @staticmethod
33         def _sigmoid(z: np.ndarray) -> np.ndarray:
34             # Clip for numerical stability (avoid overflow in exp).
35             return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37         def _standardize(self, X: np.ndarray) -> np.ndarray:
38             return (X - self.mean) / self.std
39
40         def fit(self, X, y, feature_names: Optional[List[str]] = None,
41               class_weight: str = "balanced") -> "LogisticRegression":
42             """Fit with optional balanced class weighting (rally windows are imbalanced
43             ? many spurious candidates per real rally)."""
44             X = np.asarray(X, dtype=float)
45             y = np.asarray(y, dtype=float)
46             if X.ndim != 2 or X.shape[0] == 0:
47                 raise ValueError("X must be a non-empty 2D array")
48             self.feature_names = feature_names
49
```

```

50         self.mean = X.mean(axis=0)
51         self.std = X.std(axis=0)
52         self.std[self.std == 0] = 1.0 # guard constant features
53         Xs = self._standardize(X)
54         n, d = Xs.shape
55
56         if class_weight == "balanced":
57             n_pos = max(1, int(y.sum()))
58             n_neg = max(1, int((1 - y).sum()))
59             w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60         else:
61             w_pos = w_neg = 1.0
62         sw = np.where(y == 1, w_pos, w_neg)
63
64         self.w = np.zeros(d)
65         self.b = 0.0
66         for _ in range(self.epochs):
67             p = self._sigmoid(Xs @ self.w + self.b)
68             err = (p - y) * sw
69             grad_w = Xs.T @ err / n + self.l2 * self.w
70             grad_b = float(err.mean())
71             self.w -= self.lr * grad_w
72             self.b -= self.lr * grad_b
73         return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:

```



```

84         return idx, window, []
85     try:
86         prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
87         raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
88         mapped = _offset_segments(raw, cs, ce)

```

38. assistant (2026-06-16T04:39:07.690Z)

Perfect. Now let me check training.py for the label_candidates function:

39. user (2026-06-16T04:39:08.104Z)

```

50: def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],

```

40. assistant (2026-06-16T04:39:10.457Z)

Now let me check what the critical gotchas are by examining a few more files. Let me check if there's any place where a variant has numpy/LogisticRegression used in the CONTROL path:

41. user (2026-06-16T04:39:11.188Z)

```

123:     def is_learned(self) -> bool:
158: CONTROL = Variant(name="control")
351:     "is_learned": variant.is_learned(),

```

42. assistant (2026-06-16T04:39:13.101Z)

Let me verify what the CONTROL variant actually contains and check a critical path:

43. user (2026-06-16T04:39:13.648Z)

```

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

```

44. user (2026-06-16T04:39:16.587Z)

```

156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160

```

45. assistant (2026-06-16T04:39:18.646Z)

Perfect. Now let me examine the predict() function more carefully and verify the flow for CONTROL:

46. user (2026-06-16T04:39:19.218Z)

```

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

```

47. user (2026-06-16T04:39:21.765Z)

```

213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO

```

```

221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
225         """
226         mask = _gate_mask(variant, vc)
227         if variant.feature_names:
228             if model is None:
229                 if variant.classifier_model is None:
230                     raise ExperimentError(
231                         f"variant {variant.name!r} is learned but has no model to predict "
232                         f"with (pass a fitted model or set classifier_model)")
233                     model = LogisticRegression.load(variant.classifier_model)
234             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
235             if calibrator is not None:
236                 proba = [calibrator(p) for p in proba]
237             thr = variant.threshold if threshold is None else threshold
238             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
239             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240             return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242

```

48. assistant (2026-06-16T04:39:24.188Z)

Perfect! Now let me verify how the feature_names behave in learning vs non-learning paths. Let me look more carefully at the _gate_mask and confirm what the call chain looks like:

49. user (2026-06-16T04:39:24.592Z)

```
<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>
```

50. user (2026-06-16T04:39:27.174Z)

```

200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
202         n = len(vc.intervals)
203         mask = [True] * n
204         if variant.min_crossings > 0:
205             col = _feature_column(vc, "net_crossings")
206             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207         if variant.min_players > 0:
208             col = _feature_column(vc, "players_in_court")
209             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
210         return mask
211

```

51. assistant (2026-06-16T04:39:29.693Z)

Good. Now let me understand the full data flow from a golden fixture. Let me look at what a golden feature JSON looks like:

52. user (2026-06-16T04:39:30.420Z)

(Bash completed with no output)

53. assistant (2026-06-16T04:39:32.591Z)

Let me look at fusion_golden.py to understand the structure:

54. user (2026-06-16T04:39:33.584Z)

```
14:Each output `output/<name>_golden_features.json` is the replayable feature substrate ? re-run
34:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35:# The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
118:         "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
149:         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
```

55. assistant (2026-06-16T04:39:35.833Z)

Now let me compile all the findings. Let me create a comprehensive report based on all the code I've read.

56. user (2026-06-16T04:39:36.647Z)

Chapter marked: "API mapping complete". Continue your current work.

57. assistant (2026-06-16T04:40:38.858Z)

Based on my thorough read of the golden-fixtures worktree, here is the EXACT API mapping you requested:

1. EXACT FUNCTION SIGNATURES + RETURN TYPES

****tracknet_runner.py****

```
`python
parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]
    # Returns sorted list of TrajectoryPoint(frame: int, visible: bool, x: float, y: float)
    # Parses CSV with columns: Frame, Visibility (0|1), X, Y
    # Silently skips malformed rows (KeyError/ValueError)

trajectory_to_action_windows(
    points: List[TrajectoryPoint],
    fps: float,
    frame_width: float,
    chunk_start: float = 0.0,
    velocity_thresh: float = 0.02,
    merge_gap: float = 2.0,
    min_window_duration: float = 2.0,
    chunk_index: Optional[int] = None,
    max_window_duration: float = 0.0,
    min_split_gap: float = 0.5,
) -> List[Dict[str, Any]]
    # Returns: [{
    #   "start": float (seconds),
    #   "end": float (seconds),
    #   "active_frame_count": int,
    #   "peak_velocity": float,
    #   "mean_velocity": float,
    #   "track_id_count": int (always 1),
    #   "chunk_index": Optional[int]
    # }]
    # OCCLUSION: visible==False breaks trajectory (sets prev=None, resets velocity chain)
    # CLAMPS: fps<=0 ? 30.0; frame_width<=0 ? 1280.0
    # BOUNDED WINDOWING (#187): max_window_duration > 0 triggers _split_overlong_groups()
    #   ? recursively splits windows > max_window_duration at largest inactivity gap
    #   (gap >= min_split_gap frames), never merges, only subdivides
...`
```

****windowing.py****

```
`python
```

```

@dataclass(frozen=True)
class WindowingPreset:
    name: str
    velocity_thresh: float
    merge_gap: float
    min_window_duration: float
    max_window_duration: float = 0.0

    def as_tuple(self) -> Tuple[float, float, float]
        # Returns (velocity_thresh, merge_gap, min_window_duration)

    def to_dict(self) -> Dict[str, Any]

SERVED: WindowingPreset # sourced from IndexingConfig defaults
# name="served", velocity_thresh=0.02, merge_gap=2.0,
# min_window_duration=2.0, max_window_duration=0.0 (default OFF)

PROPOSAL: WindowingPreset
# name="proposal", velocity_thresh=0.005, merge_gap=1.0,
# min_window_duration=0.5, max_window_duration=0.0

PRESETS: Dict[str, WindowingPreset] # {"served": SERVED, "proposal": PROPOSAL}

served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset
# Reads actual served config (overrides honoured)

build_windows(
    points: List[Any],
    fps: float,
    frame_width: float,
    preset: WindowingPreset
) -> List[Dict[str, Any]]
# Thin wrapper: calls TrackNetRunner.trajectory_to_action_windows with preset params

windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]
# Returns [(w["start"], w["end"]) for w in windows]
# Interval = Tuple[float, float]
...

```

****distill_local.py****

```

```python
FEATURE_NAMES: List[str] = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
PROPOSAL = (0.005, 1.0, 0.5) # velocity_thresh, merge_gap, min_window_duration
BASELINE_LOCAL = (0.02, 2.0, 2.0) # the SERVED windowing

build_candidates(
 trajectory_csv: str,
 fps: float,
 frame_width: float,
 proposal: Tuple[float, float, float] = PROPOSAL
) -> Tuple[List[Interval], List[List[float]]]
Returns (candidate_intervals, feature_rows)
Interval = Tuple[float, float]
feature_rows: each row = [duration, peak_velocity, mean_velocity, active_ratio,
net_crossings]
Net crossings sourced from rally_gate.gate_windows(..., min_crossings=0) ? .crossings field
...

```

## **\*\*experiment.py\*\***

```

```python
@dataclass
class VideoCandidates:

```

```

name: str
intervals: List[Interval]
features: Dict[str, List[float]] = field(default_factory=dict)
    # Column-major: feature_name ? list of per-candidate values
gts: Optional[List[Interval]] = None
    # Interval = Tuple[float, float]

@dataclass
class Variant:
    name: str
    segmenter: str = "wasb_hybrid"
    config_overrides: Dict[str, Any] = field(default_factory=dict)
    cue_flags: Dict[str, bool] = field(default_factory=dict)
    min_crossings: int = 0    # Gate A (net-crossing threshold)
    min_players: int = 0      # Gate B (player count threshold)
    classifier_model: Optional[str] = None    # path to frozen LogisticRegression JSON
    feature_names: Optional[List[str]] = None    # which persisted features to use
    threshold: float = 0.5    # classifier probability cutoff
    merge_gap: float = 1.0    # post-filter overlap-merge gap (seconds)
    tune_threshold: bool = False    # pick F1-best threshold on train fold
    calibrate: Optional[str] = None    # "platt" | "isotonic" | None

    def is_learned(self) -> bool
        # True iff (classifier_model is not None) OR (feature_names is not None)

    def to_dict(self) -> dict

    @classmethod
    def from_dict(cls, d: dict) -> "Variant"

    def spec_hash(self) -> str
        # Stable 12-char SHA1 hash of to_dict() (json sorted)

CONTROL = Variant(name="control")
    # Baseline: all fields at defaults (no gate, no learned filter, pure merge)
    # is_learned() ? False
    # predict(CONTROL, ...) uses ONLY _gate_mask (trivial, all True) + merge_overlaps
    # PURE STDLIB (no numpy/LogisticRegression)

def predict(
    variant: Variant,
    vc: VideoCandidates,
    model: Optional[LogisticRegression] = None,
    threshold: Optional[float] = None,
    calibrator: Optional[Callable[[float], float]] = None,
) -> List[Interval]
    # 1. _gate_mask(variant, vc) ? boolean keep-mask
    #   - if min_crossings > 0: read "net_crossings" column, apply threshold
    #   - if min_players > 0: read "players_in_court" column, apply threshold
    #   - missing columns default to [0.0] * n_candidates (logs warning)
    # 2. if variant.feature_names:
    #   - load model (supplied, or from variant.classifier_model)
    #   - predict_proba(feature_matrix) ? probs
    #   - optionally calibrate(proba)
    #   - apply threshold ? additional mask
    # 3. kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
    # 4. return merge_overlaps(kept, gap_tol=variant.merge_gap)
    ...

```

****metrics.py****

```

```python
Interval = Tuple[float, float]

@dataclass

```

```

class Score:
 iou_threshold: float
 num_pred: int
 num_gt: int
 true_positives: int
 false_positives: int
 false_negatives: int
 precision: float
 recall: float
 f1: float
 mean_iou: float
 mean_start_error: float
 mean_end_error: float

 def as_dict(self) -> dict

interval_iou(a: Interval, b: Interval) -> float
 # Temporal IoU; returns 0.0 if non-overlapping or degenerate

match_intervals(
 preds: List[Interval],
 gts: List[Interval],
 iou_threshold: float = 0.5
) -> MatchResult
 # Greedy one-to-one matching by descending IoU
 # Returns MatchResult(matches, unmatched_pred_indices, unmatched_gt_indices)

score(
 preds: List[Interval],
 gts: List[Interval],
 iou_threshold: float = 0.5,
 match_result: Optional[MatchResult] = None
) -> Score
 # Computes precision, recall, F1, mean_iou, boundary errors
...

```

#### **\*\*gt\_loader.py\*\***

```

...python
load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]
 # strict=True: raise ValueError on bad rows (corpus/gate use)
 # strict=False: skip bad rows, LOG count (legacy tuning-driver behavior)
 # Accepts both header conventions: "start"/"end" OR "start_time"/"end_time"
 # Positional fallback if no header: columns 0,1
 # Timestamps: decimal seconds or MM:SS / HH:MM:SS
 # Returns sorted by start; warns on overlapping GTs
...

```

#### **\*\*rally\_gate.py\*\***

```

...python
@dataclass
class GatedWindow:
 start: float
 end: float
 crossings: int # net crossing count
 visible_points: int
 kept: bool # crossings >= min_crossings

estimate_play_axis(points) -> Tuple[str, float, float]
 # Returns (axis: "x"|"y", net_value: float, span: float)
 # Axis = the image dimension with larger shuttle spread (play axis)
 # net_value = median of that axis (where shuttle clusters near net)
 # span = spread (p95 - p5) for that axis

```

```

count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int
 # Counts sign changes of (coord - net) across visible points
 # Ignores points within deadband of the net

gate_windows(
 points,
 windows: List[Interval],
 fps: float,
 min_crossings: int = 2,
 deadband_frac: float = 0.06,
 estimate: Optional[Tuple[str, float, float]] = None
) -> List[GatedWindow]
 # Annotates each window with crossing count and keep decision
 # estimate parameter: precomputed (axis, net, span) to avoid O(P log P) re-estimation
 # deadband = deadband_frac * span
 # Returns sorted by window start time

apply_gate(
 points,
 windows: List[Interval],
 fps: float,
 min_crossings: int = 2,
 deadband_frac: float = 0.06
) -> List[Interval]
 # Convenience: returns only windows where kept=True
...

```

#### **\*\*fusion\_compare.py\*\* (loader pattern)**

```

...python
load_videos(features_dir: str) -> List[experiment.VideoCandidates]
 # Loads every *_golden_features.json in features_dir
 # For each: reads c["shuttle"][fn] and c["person"][fn] UNCONDITIONALLY
 # (no .get() fallback ? KeyError if missing)
 # Returns VideoCandidates with features={shuttle_cols + person_cols}
...

```

#### **\*\*ablation.py\*\* (loader pattern)**

```

...python
load_videos(features_dir: str) -> List[VideoCandidates]
 # Loads every *_golden_features.json
 # For shuttle + person: reads unconditionally c["shuttle"][fn], c["person"][fn]
 # For audio: reads c.get("audio") or {} ? .get(fn, 0.0) with fallback to 0.0
 # Older JSONs without audio block: all audio columns default to [0.0]
...

```

---

## **2. CONCRETE INTEGRATION RECIPE (CONTROL-only, pure stdlib)**

```

...python

```

### **Step 1: Load and parse trajectory CSV**

```

points: List[TrajectoryPoint] = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

```

### **Step 2: Window the trajectory (use SERVED preset from windowing.py)**

```

from backend.eval.windowing import SERVED, build_windows
windows: List[Dict[str, Any]] = build_windows(points, fps=30.0, frame_width=1280.0,
preset=SERVED)

```

### **Step 3: Build candidates (extract features including net\_crossings via rally\_gate)**

```

from backend.eval.distill_local import build_candidates

```

```
intervals, X = build_candidates(trajecory_csv, fps=30.0, frame_width=1280.0)
```

**intervals: List[Interval]**

**X: List[List[float]] with 5 columns [duration, peak\_velocity, mean\_velocity, active\_ratio]**

**Step 4: Gate on net-crossings (rally\_gate.gate\_windows is called within build\_candidates)**

```
from backend.pipeline.detectors import rally_gate
```

```
gated = rally_gate.gate_windows(points, intervals, fps=30.0, min_crossings=2)
```

**gated: List[GatedWindow]**

**Step 5: Score predictions against golden labels**

```
from backend.eval.gt_loader import load_gt_intervals
```

```
gts: List[Interval] = load_gt_intervals(gt_csv_path, strict=True)
```

**Step 6: Create VideoCandidates structure for experiment harness**

```
from backend.eval.experiment import VideoCandidates, CONTROL, predict
```

```
vc = VideoCandidates(
 name="test_video",
 intervals=intervals,
 features={}, # CONTROL doesn't use features
 gts=gts
)
```

**Step 7: Predict with CONTROL variant**

```
preds: List[Interval] = predict(CONTROL, vc)
```

**predict() for CONTROL:**

- mask = [True] \* len(intervals) (no gates, feature\_names=None)
- kept = intervals (all pass)
- return merge\_overlaps(intervals, gap\_tol=1.0)

**Step 8: Score**

```
from backend.eval.metrics import score as score_metrics
```

```
result = score_metrics(preds, gts, iou_threshold=0.5)
```

**result.precision, result.recall, result.f1, result.mean\_iou,**

**result.true\_positives, result.false\_positives, result.false\_negatives**

```
...
```

```
EXACT CALL CHAIN (no branches):
```

```
...
```

```
parse_trajectory_csv(csv)
```

```
? List[TrajectoryPoint]
```

```
build_windows(points, fps, fw, SERVED)
```

```
? calls TrackNetRunner.trajectory_to_action_windows(vt=0.02, mg=2.0, mwd=2.0, max_win=0)
```

```
? List[Dict]
```

```
windows_to_intervals(windows)
```

```
? List[Interval]
```

```
gate_windows(points, intervals, fps, min_crossings=2)
```

```
? List[GatedWindow] (crossings field used by build_candidates)
```

```
predict(CONTROL, VideoCandidates)
```

```
? _gate_mask(CONTROL, vc) # all True
```

```
? merge_overlaps(intervals, gap_tol=1.0)
```

```
? List[Interval]
```

```
score(preds, gts, 0.5)
? Score object
...
```

---

### 3. RED-TEAM GOTCHAS (CONFIRMED AGAINST CODE)

**\*\*(a) Do existing loaders read GT via LENIENT vs STRICT?\***

**\*\*YES ? INCONSISTENT (NOT FIXED)\*\*** [gotcha confirmed]

- ``calibrate_wasb.load_golden_gts()`` ? calls ``load_gt_intervals(csv_path, strict=False)``
  - **\*\*File: backend/eval/calibrate\_wasb.py:41\*\***
  - Silently skips bad rows, logs count
- ``experiment.evaluate_variant()`` and ``distill_local.load_video()`` call ``load_golden_gts(spec["labels"])`` (from `calibrate_wasb`)
  - Uses lenient loader, not strict
- The strict loader ``load_gt_intervals(csv_path, strict=True)`` exists but is **\*\*NOT** used by the eval loaders\*\*
- **\*\*Impact:\*\*** A GT CSV with malformed rows (missing columns, bad timestamps) silently shrinks the GT set, flattering precision

**\*\*(b) Does experiment `\_feature\_column` silently default MISSING features to 0.0?\***

**\*\*YES\*\*** [gotcha confirmed]

- **\*\*File: backend/eval/experiment.py:180-192\*\***
- ````python`
- `def _feature_column(vc: VideoCandidates, name: str) -> List[float]:`
- `col = vc.features.get(name)`
- `n = len(vc.intervals)`
- `if col is None:`
- `logger.warning("variant references feature %r absent from %s ? treating as 0.0",`
- `name, vc.name)`
- `return [0.0] * n # SILENT ZERO FILL`
- `# ...rest`
- `````
- Logs a warning but DOES NOT raise; missing feature columns silently become 0.0
- This lets a sparse/old fixture still predict (gates + learned scorer work with zeros)

**\*\*(c) Does any loader read c['person'] UNCONDITIONALLY (KeyError on stripped fixture)?\***

**\*\*YES ? CRITICAL\*\*** [gotcha confirmed]

- **\*\*fusion\_compare.py:48\*\*** ``features[fn] = [float(c["person"][fn]) for c in cands]``
  - Direct subscript access, no ``.get()`` fallback
  - If a fixture has NO "person" key in any candidate, **\*\*KeyError\*\*** on the first loop
- **\*\*File: backend/eval/fusion\_compare.py:35-52\*\***
- **\*\*ablation.py:401\*\*** Same pattern: ``features[fn] = [float(c["person"][fn]) for c in cands]``
  - **\*\*File: backend/eval/ablation.py:386-407\*\***
- **\*\*ablation.py:403\*\*** Audio HAS defensive ``.get()`: `c.get("audio") or {}.get(fn, 0.0)``
  - Audio defaults to 0.0 if missing; person does NOT
- **\*\*Impact:\*\*** A fixture with person-cue stripped (e.g., for privacy) will crash both `fusion_compare` and `ablation` at load time

**\*\*(d) Does CONTROL avoid numpy/LogisticRegression entirely (pure stdlib)? Which variants are is\_learned()?\***

**\*\*YES ? CONTROL is pure stdlib; learned variants use numpy/LogisticRegression\*\*** [gotcha confirmed]

- **\*\*CONTROL definition (backend/eval/experiment.py:158)\*\*:**
- ````python`
- `CONTROL = Variant(name="control") # all fields at defaults`
- `````
- ``feature_names=None`, `classifier_model=None`, `min_crossings=0`, `min_players=0``
- ``is_learned()` ? `False``

- ``predict(CONTROL, vc)`` ? calls ``_gate_mask`` (stdlib only) + ``merge_overlaps`` (stdlib only)
- **\*\*NO numpy, NO LogisticRegression\*\***
- **\*\*is\_learned() definition (backend/eval/experiment.py:123-125)\*\*:**

```

`python
def is_learned(self) -> bool:
 return self.classifier_model is not None or bool(self.feature_names)
`

```

  - Learned: any variant with ``classifier_model`` set OR ``feature_names`` set
  - Learned variants call ``LogisticRegression.load()`` (requires numpy)
  - **\*\*File: backend/eval/experiment.py:233\*\*** ``model =`  
`LogisticRegression.load(variant.classifier_model)``
  - **\*\*File: backend/eval/experiment.py:234\*\*** ``proba = [float(p) for p in`  
`model.predict_proba(...)]``
- **\*\*Examples of is\_learned() variants\*\*:**
  - ``fusion_compare.shuttle_only``: ``feature_names=list(SHUTTLE_FEATURE_NAMES)``
  - ``ablation.learned_full_variant()``: ``feature_names=[all 13 columns]``
  - Any variant with ``classifier_model="path/to/model.json"`` set

---

## SUMMARY

All exact APIs mapped. The CONTROL variant is production-ready stdlib code (pure Python, no ML deps). The integration recipe is deterministic and reproducible: trajectory CSV ? windows ? candidates ? `predict(CONTROL)` ? `score(preds, gts)`. Three critical loader gotchas confirmed: (a) lenient GT reading flattens precision, (b) missing features silently become 0.0, (c) **\*\*person cue strips cause KeyError on fusion\_compare/ablation\*\*** ? this is the highest-risk one for your ``replay_fixture()`` if feeding a CSV with no person data. CONTROL avoids all ML entirely.